

CodeInsight-AI: Advanced Architecture & Feature Specifications

Comprehensive guide to the newly implemented AI RAG (pgvector), Real-Time SSE Streaming, Token-Bucket Rate Limiting, and Production Cloud Architecture.

1. System Architecture & Production Deployment

CodeInsight-AI is an enterprise-grade developer analytics and AI-powered coding interview coach. The application has been elevated from a monolithic local prototype into a high-performance, distributed, production-ready cloud system.

Tier	Technology Stack	Production Hosting	Key Capabilities
Frontend	React 18 + Vite + TypeScript	Render Global CDN	Real-time SSE event consumption, responsive Recharts dashboards, animated typewriter markdown.
Backend	Spring Boot 3.3.5 + Java 21	Render Docker (US East)	Stateless JWT auth, Bucket4j token bucket rate limiting, automated JDBC URI normalizer.
Database	Serverless PostgreSQL 16	Neon Cloud (Ohio AWS)	Native pgvector extension, 1536-dimensional vector cosine distance search.
AI / LLM	OpenAI GPT-4o-mini + RAG	Cloud API + Local Fallback	Retrieval-Augmented Generation, chunk-by-chunk HTTP event streaming.

2. Feature 1: AI RAG with PostgreSQL pgvector

The Problem: Standard Large Language Models frequently suffer from hallucinations—fabricating problem names, broken URLs, or generic guidance (e.g. *'practice more dynamic programming'*). Furthermore, traditional keyword database search fails when queries express concepts rather than literal words.

The Solution: Neon PostgreSQL with native pgvector turns the relational database into a semantic vector similarity engine. Every curated DSA problem is represented as a high-dimensional vector in 1536-dimensional Euclidean space.

- **Cosine Similarity Search:** Employs PostgreSQL's native vector cosine distance operator (`<=>`) to retrieve the closest matching problems mathematically: `1.0 - (embedding <=> CAST(? AS vector))`.
- **Curated Problem Library:** Pre-seeded with 30+ top interview coding problems from LeetCode, CodeChef, and GeeksforGeeks across 8 core paradigms (Tree DP, Graph BFS/DFS, Monotonic Stack, Sliding Window, Two Pointers, etc.).

- **Cognitive Gap Augmentation:** Instead of blindly guessing, the system evaluates the candidate's actual failed topics, generates a targeted query vector, retrieves the 4 most relevant authentic questions, and injects verified problem links directly into the LLM context prompt.
- **Zero Hallucinations:** The AI is strictly anchored to genuine problems with live platform URLs. Even in offline mode or during API limits, the RAG ranking delivers verified real problems.

3. Feature 2: Real-Time Server-Sent Events (SSE) Streaming

The Problem: Generating a comprehensive multi-week study plan or interview roadmap takes 10 to 20 seconds. Traditional synchronous REST APIs force users to stare at a frozen spinner. On free cloud hosts (Render/Cloudflare), long HTTP calls are also at constant risk of 30-second gateway timeouts.

The Solution: Implemented a reactive HTTP streaming pipeline via Spring's SseEmitter and HTML5 EventSource. As soon as the first word is generated (within ~300ms), it is piped directly to the client browser in real time.

Performance Metric	Without Streaming (Legacy)	With Real-Time SSE (Current)
Time to First Word	10 – 20 seconds (Full wait)	< 300 milliseconds (Instantaneous)
User Experience	Static blocking loading spinner	ChatGPT-style live typewriter effect
HTTP Timeout Vulnerability	High risk on Render 30s timeout	Zero risk (Continuous keep-alive stream)
Database Persistence	Saves only after entire payload finishes	Asynchronous auto-save upon stream termination

4. Feature 3: Token Bucket Rate Limiting (Bucket4j)

To protect expensive AI APIs and prevent third-party platform API abuse, CodeInsight-AI features a multi-tiered in-memory rate limiting engine implemented via **Bucket4j**. Tokens automatically refill based on a continuous bandwidth schedule.

API Endpoint Pattern	Limit Policy	Refill Schedule	Protection Objective
/api/ai/**	10 requests / min	10 tokens every 1 minute	Protects OpenAI API usage & token budgets.
/api/platforms/refresh	5 requests / 5 min	5 tokens every 5 minutes	Prevents rate bans from LeetCode / GitHub / GFG scraping.
/api/auth/**	10 requests / min	10 tokens every 1 minute	Thwarts brute-force credential stuffing and DoS attacks.
/api/** (Default)	100 requests / min	100 tokens every 1 minute	Protects general application server availability.

Standard HTTP 429 Responses: When a client consumes all bucket tokens, the filter immediately rejects requests with HTTP status 429 Too Many Requests, accompanied by standard RFC headers: Retry-After, X-Rate-Limit-Remaining, and X-Rate-Limit-Limit.

5. Feature 4: Resilient Caching & Cloud Connection Engine

Smart Multi-Tier Cache Fallback: Configured via Spring Cache abstraction. When an enterprise Redis cluster is available, all platform analytics and solved problem counts are cached with a 30-minute TTL. If Redis is absent or restarts, the custom RedisConfig catches connection exceptions gracefully and automatically activates an in-memory ConcurrentMapCacheManager with zero downtime.

Cloud JDBC URL Normalizer: Standard cloud platforms (Render, Heroku, Railway, Neon) supply database URIs formatted as postgresql://user:password@host:port/database. Modern PostgreSQL JDBC drivers reject embedded credentials in the host string. Our newly implemented DataSourceConfig interceptor parses the URI, extracts credentials, and standardizes the connection string to jdbc:postgresql://host:port/database automatically.

6. Live Production Environment Reference

Resource	Production Endpoint / Location	Status
Frontend Application	https://codeinsight-frontend-d22s.onrender.com	LIVE (200 OK)
Backend REST & SSE API	https://codeinsight-ai-backend-ncxn.onrender.com	LIVE (200 OK)
Backend Health Check	https://codeinsight-ai-backend-ncxn.onrender.com/api/auth/health	HEALTHY (UP)
Vector Database	Neon Serverless PostgreSQL 16 (AWS us-east-2)	CONNECTED

Key Takeaway: *With Phase 1 and Phase 2 fully deployed, CodeInsight-AI now features enterprise-grade stability, zero-hallucination semantic problem recommendations, sub-second perceived response times through real-time streaming, and robust API protection against rate limits.*